



PYTHON NOTES — FUNCTIONS MASTER GUIDE



TOPIC: FUNCTIONS

Functions are reusable blocks of Python code that perform a particular task.

Instead of writing the same code repeatedly, we can place it inside a function and call the function whenever we need it.

1. WHAT IS A FUNCTION?

A function is a reusable block of Python code that performs a task.

Basic example

```
def greet():  
    print('Hello!')
```

To run the function:

```
greet()
```

Output

```
Hello!
```

Important

Defining a function does **not** run it.

The function runs when we **call** it.

2. FUNCTION DEFINITION

A function is created using the `def` keyword.

```
def greet():  
    print('Hello!')
```

The parts are:

- `def` → tells Python that we are defining a function
- `greet` → the function name
- `()` → contains the function's parameters
- `:` → begins the function body
- Indented code → the instructions the function performs

3. CALLING A FUNCTION

Calling a function means telling Python to execute it.

```
greet()
```

A function can be called multiple times:

```
greet()  
greet()  
greet()
```

The same function can therefore be reused.

4. PARAMETERS

A **parameter** is a variable written inside the parentheses when we define a function.

Example:

```
def greet(name, age):  
    print('My name is', name, 'and I am', age, 'years old')
```

Here:

```
name  
age
```

are **parameters**.

They are placeholders waiting to receive values.

5. ARGUMENTS

An **argument** is the actual value that we give to a function when we call it.

Example:

```
def greet(name, age):  
    print('My name is', name, 'and I am', age, 'years old')  
  
greet('James', 12)
```

Here:

```
'James '  
12
```

are **arguments**.

Easy way to remember

Parameter = placeholder

Argument = actual value

6. PARAMETERS VS ARGUMENTS

Parameter

Appears in the **function definition**.

```
def greet(name, age):
```

`name` and `age` are parameters.

Argument

Appears in the **function call**.

```
greet('James', 12)
```

`'James'` and `12` are arguments.

Quick comparison

Term	Where it appears	Example
Parameter	Function definition	<code>name</code>
Argument	Function call	<code>'James'</code>

7. POSITIONAL ARGUMENTS

Positional arguments are arguments matched according to their **position**.

Example:

```
def greet(name, age):  
    print(name, age)  
  
greet('James', 12)
```

Python matches them like this:

```
First argument → first parameter  
'James'       → name  
  
Second argument → second parameter  
12             → age
```

Important

With positional arguments, **order matters**.

This:

```
greet('James', 12)
```

means:

```
name = 'James'  
age = 12
```

But:

```
greet(12, 'James')
```

means:

```
name = 12  
age = 'James'
```

Python follows the position.

8. POSITIONAL PARAMETERS

The phrase **positional parameter** is sometimes used informally to describe a parameter that receives a value according to its position.

For our current level, the most useful idea is:

Positional argument → the value is supplied by position.

The parameters are the placeholders that receive those values.

9. KEYWORD ARGUMENTS

Keyword arguments are arguments where we explicitly give the **parameter name**.

Example:

```
def greet(name, age):  
    print('My name is', name, 'and I am', age, 'years old')  
  
greet(name='Uche', age=19)
```

Here:

```
name='Uche'  
age=19
```

are keyword arguments.

Python knows exactly which parameter receives each value.

10. KEYWORD ARGUMENTS CAN BE IN A DIFFERENT ORDER

Because keyword arguments include the parameter names, their order does not have to match the order in the function definition.

Function definition:

```
def greet(name, age):  
    print(name, age)
```

We can call:

```
greet(age=11, name='Linda')
```

This still works.

Python matches:

```
age → 11  
name → Linda
```

It does not depend on their order because the names are provided.

11. POSITIONAL VS KEYWORD ARGUMENTS

Positional

```
greet('James', 12)
```

The order matters.

Keyword

```
greet(name='James', age=12)
```

The parameter names are provided.

The order can be changed:

```
greet(age=12, name='James')
```

12. DEFAULT PARAMETERS

A default parameter already has a value in the function definition.

Example:

```
def student_info(name='Katie', age=13):  
    print(name, age)
```

Now we can call:

```
student_info()
```

Python uses:

```
name = Katie  
age = 13
```

We can also replace the defaults:

```
student_info(name='Chris', age=14)
```

13. RETURN

The `return` statement sends a value back from a function.

Example:

```
def add_numbers(a, b):  
    return a + b
```

We can store the returned value:

```
result = add_numbers(5, 3)  
print(result)
```

Output:

```
8
```

`print()` vs `return`

`print()` displays something.

`return` sends a value back so the program can use it later.

14. LOCAL VARIABLES

A local variable is a variable created inside a function.

Example:


```
def greet():  
    message = 'Hello'  
    print(message)
```

`message` exists inside the function.

It normally cannot be used directly outside that function.

15. GLOBAL VARIABLES

A global variable is created outside a function.

Example:

```
name = 'Christabel'  
  
def greet():  
    print(name)  
  
greet()
```

The function can access the global variable.

Important

Local and global variables have different scopes.

Local → inside the function

Global → outside the function

16. `*args`

`*args` allows a function to accept an **unknown number of positional arguments**.

Example:

```
def add_numbers(*args):  
    total = sum(args)
```

```
print(total)

add_numbers(5, 6, 7)
```

Output:

```
18
```

The arguments:

```
5, 6, 7
```

are collected into `args`.

`args` is a **tuple**.

17. USING `*args` WITH `sum()`

```
def add_numbers(*args):
    print(sum(args))

add_numbers(10, 20, 30)
```

Output:

```
60
```

18. USING `*args` WITH `max()`

```
def find_biggest(*numbers):
    biggest = max(numbers)
    print(biggest)

find_biggest(1, 100, 21, 50)
```

Output:

100

`max()` finds the **largest value**.

Status: 🟡 Introduced through `*args` practice; not yet fully taught as its own built-in-function lesson.

19. USING `*args` WITH LOOPS

We can loop through the values collected by `*args`.

```
def show_fruits(*fruits):  
    for fruit in fruits:  
        print(fruit)  
  
show_fruits('Banana', 'Apple', 'Mango', 'Cherry')
```

Output:

```
Banana  
Apple  
Mango  
Cherry
```

20. BUILT-IN FUNCTIONS USED WITH `*args`

`*args` collects values into a tuple, so we can use Python's built-in functions to work with those values.

`sum()`

`sum()` adds numbers together.

```
numbers = (5, 10, 15)  
print(sum(numbers))
```

Output:

30

Status: 🟡 Used in practice; should be reviewed properly.

`max()`

`max()` finds the largest value.

```
numbers = (5, 10, 15)
print(max(numbers))
```

Output:

15

Status: 🟡 Introduced, not yet properly taught.

`min()`

`min()` finds the smallest value.

Example:

```
numbers = (8, 3, 15, 2)
print(min(numbers))
```

Output:

2

Status: 🟡 Introduced, not yet properly taught.

`len()`

`len()` tells us how many items are in a collection or how many characters are in a string.

Example:

```
skills = ('Python', 'HTML', 'CSS')
print(len(skills))
```

Output:

3

With `*args`:

```
def count_items(*items):
    print(len(items))

count_items('Python', 'HTML', 'CSS', 'JavaScript')
```

Output:

4

Status: 🟡 Introduced, not yet properly taught.

21. QUICK COMPARISON OF THESE BUILT-IN FUNCTIONS

```
sum() → adds values
max() → largest value
min() → smallest value
len() → number of items / characters
```

These functions can be useful when working with `*args`, lists, tuples, and strings.

22. NORMAL PARAMETERS WITH `*args`

A function can have normal parameters before `*args`.

Example:

```
def greet_people(greeting, *names):  
    for name in names:  
        print(greeting, name)  
  
greet_people('Hello', 'Chris', 'Joyce', 'James')
```

Here:

```
greeting → normal parameter  
names → collects extra positional arguments
```

23. ****kwargs**

****kwargs** allows a function to accept an **unknown number of keyword arguments**.

Example:

```
def show_info(**details):  
    print(details)  
  
show_info(name='Christabel', age=14, country='Nigeria')
```

The keyword arguments are collected into a **dictionary**.

The result looks like:

```
{'name': 'Christabel', 'age': 14, 'country': 'Nigeria'}
```

Important

kwargs is only a common name.

We could also write:

```
def show_info(**details):
```

The name does not have to be `kwargs`.

The important part is the `**`.

24. USING `**kwargs` WITH A LOOP

Example:

```
def student_details(**details):
    for key, value in details.items():
        print(key, ': ', value)

student_details(
    name='Joyce',
    age=13,
    school='Ludi School'
)
```

Output:

```
name : Joyce
age : 13
school : Ludi School
```

25. `*args` VS `**kwargs`

This is very important.

`*args`

Collects extra **positional arguments**.

```
show_fruits('Banana', 'Apple', 'Mango')
```

The values are collected into a **tuple**.

`**kwargs`

Collects extra **keyword arguments**.

```
student_details(name='Joyce', age=13)
```

The values are collected into a **dictionary**.

Easy memory trick

```
*args    → positional → tuple  
**kwargs → keyword   → dictionary
```

26. USING `*args` AND `**kwargs` TOGETHER

A function can use both.

Example:

```
def display_data(*args, **kwargs):  
    print('Positional:', args)  
    print('Keyword:', kwargs)  
  
display_data(  
    10,  
    'Chris',  
    name='Belle',  
    age=14  
)
```

Output:

```
Positional: (10, 'Chris')  
Keyword: {'name': 'Belle', 'age': 14}
```

So:

```
10, 'Chris' → *args  
name='Belle', age=14 → **kwargs
```

27. USING INPUT WITH `*args`

`input()` can collect values which are then passed as positional arguments.

Example:

```
def favorite_foods(*foods):
    for food in foods:
        print(food)

food1 = input('Enter a food: ')
food2 = input('Enter another food: ')
food3 = input('Enter one more food: ')

favorite_foods(food1, food2, food3)
```

The flow is:

```
input()
↓
variables
↓
arguments
↓
*args
↓
function
```

28. USING INPUT WITH `**kwargs`

Input can also be passed as keyword arguments.

Example:

```
def student_info(**details):
    for key, value in details.items():
        print(key, ': ', value)

name = input('Enter your name: ')
age = input('Enter your age: ')
```

```
student_info(name=name, age=age)
```

29. F-STRINGS

An f-string is a convenient way to place variables directly inside a string.

Example:

```
name = 'Katie'  
age = 14  
  
print(f'My name is {name}. I am {age} years old.')
```

Output:

```
My name is Katie. I am 14 years old.
```

Important

The string begins with `f`:

```
f'...'
```

Variables go inside curly braces:

```
{name}  
{age}
```

Incorrect

```
print(f'My name is [name].')
```

Correct

```
print(f'My name is {name}.')
```

30. OUR FINAL CAREER PROFILE PROJECT

We combined many Python concepts into one program.

The program can:

- Ask for the user's name
- Ask for a career
- Accept any number of skills
- Ask for school information
- Ask for certificate information
- Clear the terminal
- Display a clean final profile

Example structure:

```
import os

def career_profile(name, career, *skills, **details):
    print(f'My name is {name}.')
    print(f'I want to become a {career}.')

    print('My skills are:')
    for skill in skills:
        print('-', skill)

    print('Other details:')
    for key, value in details.items():
        print(f'{key}: {value}')

name = input('Enter your name: ')
career = input('Enter your career: ')

skills = []

while True:
    skill = input('Enter a skill or type "done" to finish: ')

    if skill.lower() == 'done':
        break

    skills.append(skill)

school = input('Enter your school: ')
certi = input('Enter your school leaving certificate details: ')
```

```
os.system('cls' if os.name == 'nt' else 'clear')

career_profile(
    name,
    career,
    *skills,
    school=school,
    certi=certi
)
```

This project combined:

- Functions
- Parameters
- Arguments
- Positional arguments
- Keyword arguments
- `*args`
- `**kwargs`
- `input()`
- Lists
- `while` loops
- `for` loops
- `if`
- `break`
- `.append()`
- f-strings
- `import`
- `os`

31. TOPICS WE USED BUT HAVE NOT FULLY STUDIED YET

Some things appeared naturally in our projects but belong to other Python topics or need their own proper lesson.

`.lower()`

Used to convert a string to lowercase.

```
'done'.lower()
```

We used it in the skills loop so that `DONE`, `Done`, and `done` could be treated the same.

Belongs under: Strings

Status:  Introduced, not yet fully taught.

```
.strip()
```

Used to remove extra spaces from the beginning and end of a string.

```
' done '.strip()
```

Belongs under: Strings

Status:  Introduced, not yet fully taught.

```
.items()
```

A dictionary method that gives access to keys and values together.

```
details.items()
```

Belongs under: Dictionaries

Status:  Introduced through `**kwargs`, not yet fully taught under Dictionaries.

```
import os and os.system()
```

Used to work with the operating system and, in our project, clear the terminal.

```
import os  
  
os.system('cls' if os.name == 'nt' else 'clear')
```

Belongs under: Modules / Standard Library

Status: 🟡 Introduced, not yet fully taught.

Built-in functions we have encountered

We have used or mentioned:

```
sum() → adds values
max() → finds the largest value
min() → finds the smallest value
len() → counts items or characters
```

Important: Using these in a Functions project does **not** mean the built-in functions themselves have been fully taught.

Status:

`sum()` → 🟡 Used, needs proper review

`max()` → 🟡 Introduced, not yet fully taught

`min()` → 🟡 Introduced, not yet fully taught

`len()` → 🟡 Introduced, not yet fully taught

32. IMPORTANT TERMS TO REMEMBER

Parameter

A placeholder in a function definition.

```
def greet(name):
```

`name` is a parameter.

Argument

An actual value passed into a function.

```
greet('Chris')
```

'Chris' is an argument.

Positional argument

An argument matched according to position.

```
greet('Chris', 14)
```

Keyword argument

An argument matched using the parameter name.

```
greet(name='Chris', age=14)
```

Default parameter

A parameter with a default value.

```
def greet(name='Chris'):
```

`*args`

Extra positional arguments → tuple.

`**kwargs`

Extra keyword arguments → dictionary.

f-string

A string that allows variables to be inserted using `{}`.

FUNCTIONS PROGRESS

Current Level: Functions

Completed

- ✓ Function definitions
- ✓ Calling functions
- ✓ Parameters
- ✓ Arguments
- ✓ Parameters vs arguments
- ✓ Positional arguments
- ✓ Keyword arguments
- ✓ Positional vs keyword arguments
- ✓ Default parameters
- ✓ `return`
- ✓ Local variables
- ✓ Global variables
- ✓ `*args`
- ✓ `**kwargs`
- ✓ `input()` with functions
- ✓ f-strings
- ✓ Combining multiple function concepts
- ✓ Final Career Profile project

Introduced but not fully taught

- `sum()`
- `max()`
- `min()`
- `len()`
- `.lower()`
- `.strip()`
- `.items()`
- `import os` / `os.system()`



FUNCTIONS COMPLETE

I have completed the **core Functions topic**.

I can now:

- ✓ Create reusable functions
- ✓ Define parameters
- ✓ Pass arguments
- ✓ Use positional arguments
- ✓ Use keyword arguments
- ✓ Use default parameters
- ✓ Return values

- ✓ Understand local and global variables
- ✓ Use `*args`
- ✓ Use `**kwargs`
- ✓ Work with input inside function-based programs
- ✓ Use f-strings
- ✓ Combine functions with loops, lists, conditions, and input
- ✓ Build flexible programs instead of fixed ones

The remaining yellow items will be taught under their proper topics so that I do not confuse **"used"** with **"fully learned."**

🐍 **FUNCTIONS** → ✓ **CORE TOPIC COMPLETE**